

生成臨床 UI の更新における層間整合性の評価

ANONYMOUS AUTHOR(S)

診療を支える生成 UI では、SQL と画面が個別に動くだけでなく、診療場面に応じて指定した患者、項目、期間、情報源と、実際の表示内容との対応が一貫している必要がある。本研究では、情報要求、SQL、実行結果、表示の対応が途中で崩れた状態を層間不整合と定義し、それを見つけ、場所を示し、直せるかを調べる評価方法を示す。EHRSQL-2024 と MIMIC-IV Demo を使い、8 種類の不整合を再現可能な手順で作った。局所検査、成果物内検査、グラフで層間を照合する検査、同じ内容を表で照合する検査を比較した。134 種類の質問テンプレートから 763 組の更新前後ペアを作り、6,104 回検査した。局所検査は不整合をすべて受理し、成果物内検査も 48.10%を受理した。二つの層間検査は不整合をすべて拒否し、妥当な更新をすべて受理した。両者の差は 0 だった。検出を支えたのは保存形式ではなく、シナリオに基づく情報要求から表示までの対応関係を、実行できる検査規則として持たせたことだった。本評価は利用者への効果を直接測っていない。そこで、今回の検査を妥当な臨床 UX を評価する前提に位置付け、情報がどこから来たかを確認できる設計を論じる。

CCS Concepts: • **Human-centered computing** → **Human computer interaction (HCI)**; *Interactive systems and tools*.

Additional Key Words and Phrases: 生成ユーザーインターフェース, データ駆動型 UI, 電子カルテ, 整合性検査, 増分更新, 追跡可能性

ACM Reference Format:

Anonymous Author(s). 2027. 生成臨床 UI の更新における層間整合性の評価. In *Proceedings of Proceedings of the 2027 CHI Conference on Human Factors in Computing Systems (CHI '27)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 はじめに

医療者が必要とする画面は、診療場面によって変わる。術前外来では一時点の幅広い情報をまとめ、術後フォローでは限られた項目の推移を追う。同じ電子カルテでも、必要な項目、時間範囲、情報源、表示方法は様々ではない。生成 UI には、SQL を作るだけでなく、診療タスクに必要な情報を確認しやすい形で医療者へ返す役割がある。

この目的に対し、従来の技術検査では確かめられない部分がある。SQL の構文と実行、結果形式、UI コード、画面描画は、それぞれ個別に確認できる。しかし、各成果物が正常でも、SQL だけが別の患者を参照する、期間条件だけが外れる、更新前の結果が残る、表示部品が別のデータを参照するといった誤りは起こり得る。SQL の正答率が上がっても、その後の実行、接続、表示まで同じ要求が保たれなければ、医療者へ返る情報は正しくない。

本稿では、診療場面から定めた情報要求と、最終的に表示される情報の対応が途中で崩れた状態を**層間不整合**と呼ぶ。生成臨床 UI の品質を、情報要求から表示までの一連の流れとして捉える。ここでいう正しさは、医学的判断全体の正しさではない。あらかじめ定めた患者、項目、期間、集約、情報源、表示方法、接続、版が、更新後も一致していることを指す。

生成・編集可能な UI の研究では、タスクモデルや意味表現を介して自然言語要求と UI を結ぶ方法が示されている。Jelly は、変化する情報タスクをタスク駆動データモデルとして保持し、自然言語と直接操作による更新を支える [1]。Bridging Gulfs は、設計意図の意味表現を利用者の入力と生成 UI の間に置き、解釈を確かめながら生成できるようにする [8]。SimStep は複数のタスクレベル表現を使い、生成結果の前提を段階的に確認・修正できるようにする [5]。これらは中間表現の価値を示す一方、データ取得を伴う UI の増分更新で、情報要求から表示までのどの関係が壊れ、どの検査で見つかるかを同一候補上で比較していない。

2027. Manuscript submitted to ACM

Manuscript submitted to ACM

そこで本研究は、生成臨床 UI の更新で起こる層間不整合を調べる評価基盤を作った。診療上の情報要求と正解 SQL には、EHRSQL-2024[6] を用いる。正解 SQL を公開デモ用の MIMIC-IV Clinical Database Demo v2.2[4] 上で実行し、基準となるデータ駆動型 UI を作る。そこへ対象患者、対象項目、期間、集約方法、情報源、表示方法、データと表示の接続、更新後に残る古い結果という 8 種類の不整合を一つずつ入れる。候補を作る処理とは別の方法で正解を判定し、同じ候補を 4 種類の検査条件に入力する。この評価は、妥当な臨床 UX を生成する過程のうち、要求どおりの情報が表示へ届くという必要条件を測る。

本研究では次の問いに答える。

RQ1 定義した 8 種類のうち、SQL や表示部品を一つずつ確認しても通過する情報の誤りはどれか。

RQ2 局所検査、成果物内検査、質問から表示までをまとめて照合する検査では、不整合の見逃し、妥当な更新の受理、問題箇所の特定制がどう異なるか。

RQ3 検査で特定した問題を一回だけ自動修復すると、どの範囲を直せるか。

固定した評価方法でテスト分割を一度だけ実行した結果、134 種類の質問テンプレートから 763 組の更新前後ペアを作った。妥当候補と不整合候補を 4 条件で検査し、合計 6,104 回の結果を得た。局所検査は不整合 763 件をすべて受理し、成果物内検査も 367 件を受理した。質問から表示までをグラフで照合する検査と、同じ対応内容を一枚の表で照合する検査は、不整合をすべて拒否し、妥当候補をすべて受理した。両者は、判定、問題箇所の特定、一回の修復で同じ結果だった。この結果は、グラフという保存形式が検出に有利だという予想を支持しない。結果を分けたのは、層をまたぐ意味の対応を明示し、実行できる規則として検査したことだった。

本稿の貢献は次の五点である。

- 診療シナリオごとに情報要求と妥当な表示が変わることを踏まえ、生成臨床 UI の評価範囲を、タスク定義の医学的妥当性、層間整合性、利用者による確認に分けた。
- データ駆動型 UI の増分更新で起こる情報の誤りを、患者、項目、期間、集約、情報源、表示方法、接続、古い結果の 8 種類に分け、実験で一つずつ変えられるようにした。
- 公開デモデータ上で、同じ入力から同じ不整合を作れる手順と、候補を作る処理から独立したオラクルを組み合わせて、評価を再現できるようにした。
- 同じ候補に 4 種類の検査条件を適用し、個別成果物の妥当性だけでは層間整合性を保証できない範囲を定量化した。
- 同じ内容をグラフと表で照合したところ、結果は同じだった。この結果から、生成 UI に必要なのは特定の保存形式ではなく、検査できる層間関係だという設計上の示唆を得た。

本評価は技術評価であり、診療シナリオ自体の医学的妥当性、臨床上の安全性、診療判断の正確さ、使いやすさ、認知負荷、臨床転帰を評価していない。この境界を保ちつつ、情報抽出と表示の整合性が、利用者に価値をもたらすための前提であることを論じる。

2 関連研究

2.1 生成・編集可能な UI

生成 UI の研究は、自然言語や例示から画面を生成する段階から、生成物を利用者が理解し、編集し、再利用できる段階へ広がっている。DirectGPT は、会話に加えて生成物そのものを直接操作し、参照解決を支えることで、生成 AI との反復的な作業を扱った [7]。DynaVis は、可視化の生成と編集を対話的に行い、生成された UI 上の操作を自然言語対話と組み合わせた [10]。これらの研究は、生成物を継続的に変更できる対話対象として扱う。

更新を支えるため、中間表現を導入する研究もある。Jelly は、タスク駆動データモデルを生成 UI の背後に置き、タスクの変化をデータと画面に反映する [1]。Bridging Gulfs は、意味表現を通じて利用者の意図、生成モデ

ルの解釈、出力 UI の隔たりを小さくする [8]。SimStep は、概念、シナリオ、学習目標、UI のグラフを分け、人が各段階の仮定を編集できるようにする [5]。これらの設計では、中間表現が検査や修正の接点になる。

本研究は、より狭い問いを扱う。更新の前後で、データの意味上の対応が保たれたかを比べる。中間表現を利用するが、グラフ形式そのものは貢献として扱わない。同じ関係を一枚の表でも表し、検出結果を比較することで、保存形式と検査内容を分けて評価する。

2.2 人と生成システムの協働における検査と修正

生成システムの利用者は、出力を単純に採用または拒否するだけではない。Ignore, Trust, or Negotiate は、生成 AI の提案を利用者が無視、信頼、交渉しながら取り込む過程を示し、提案の不確かさや根拠を扱える相互作用の必要性を論じた [9]。生成 UI でも、エラー表示だけで更新全体を拒否するより、どの関係が壊れ、何を修正すると影響がどこへ及ぶかを提示する方が、利用者の判断を支えられる可能性がある。

デバッグ支援では、失敗の有無だけでなく、原因にたどり着けることが重要である。本研究は、検出、問題箇所の特定、一回の自動修復を別々に測る。これにより、拒否率だけが低い検査と、修正可能な診断を返す検査を区別する。ただし、本稿は診断表示を人が理解できるかを評価していない。技術的に特定できることと、人の判断を支えられることは別の研究課題として扱う。

2.3 自然言語質問からデータ取得と表示まで

Text-to-SQL 研究は、自然言語の質問から実行できる SQL を生成し、質問へ正しく答えられるかを評価してきた。EHRSQL-2024 は電子カルテを対象に、回答可能な質問と回答不能な質問を含む共有タスクを提供する [6]。この評価では、質問に対応する SQL を生成できるか、答えられない質問で棄権できるかが中心になる。

本研究は SQL 生成の性能を比較しない。正解 SQL を基準として使い、その後に生じる層間不整合を扱う。正しい SQL が一度得られても、UI の更新時には、質問、SQL、結果、表示のいずれかだけが別の版に更新される可能性がある。また、変更後の SQL を実行できても、対象患者や期間が要求とずれる場合がある。したがって、質問と SQL の対応に加えて、SQL と実行結果、結果と表示方法、データ取得と表示部品の接続、更新版の一致を検査する必要がある。

2.4 モデル変換と整合性

構造化データとビューの更新を対応付ける問題は、双方向変換の研究で扱われてきた。Lens は、元データとビューの往復変換が満たす性質を定式化する [2]。また、完全な正解出力を列挙しにくいモデル変換に対し、不変関係を使うメタモルフィックテストが提案されている [3]。本研究も完成した UI JSON の完全一致を求めず、変更すべき関係と保持すべき関係を述語として表す。

従来の整合性検査と本研究の違いは、生成 UI の更新を評価する際に、利用者の自然言語要求、データ取得、実行結果、表示の境界を一つの評価単位に含める点にある。完全一致を避けることで、表示文言や安定 ID など意味を変えない差を許しつつ、見た目には現れない患者や期間のずれを不整合として扱う。

3 臨床的な問題設定と評価範囲

3.1 シナリオごとに変わる情報要求

本研究の着想につながった設計過程では、麻酔科術前外来と甲状腺術後フォローを診療シナリオとして整理した。麻酔科術前外来では、手術日を基準に患者背景、術式、生活歴、アレルギー歴、既往歴、治療歴、直近の検査、麻酔歴を集約し、最後に麻酔リスクをまとめる。情報の中心は、一回の術前評価に必要な最新状態である。甲状腺

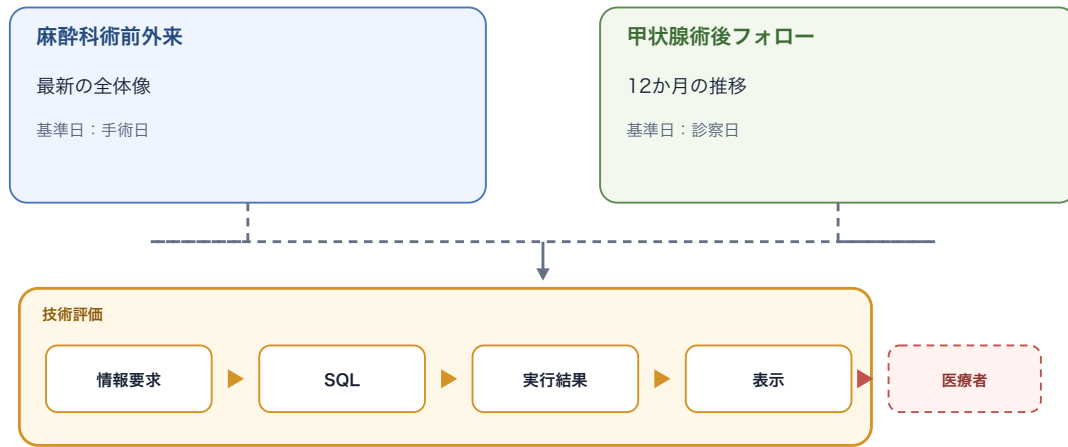


Fig. 1. 診療シナリオによって必要な情報と表示が変わる例。術前外来（左）は手術日を基準に、患者背景、手術情報、直近検査の最新の状態をまとめる。甲状腺術後フォロー（右）は診察日を基準に、甲状腺機能と補充量の12か月の推移に超音波所見を組み合わせる。本稿は情報要求から表示までを技術評価し、医学的妥当性と医療者への効果は別に評価する。二つのシナリオは正式評価ケースではない。

腺術後フォローでは、診察日を基準に甲状腺機能、超音波所見、チラーゼン補充量を確認する。甲状腺機能と補充量には直近12か月の推移が必要であり、最新値だけを示す部分と経時変化を示す部分を使い分ける。

図1は、この違いと本稿の評価範囲を示す。二つのシナリオは、同じ電子カルテでも、抽出すべき情報と表示方法が診療場面によって変わることを説明するための設計例である。正式評価の正解データには使っておらず、現在のタスクモデルも専門家の確認を終えた臨床基準ではない。

3.2 利用者価値までの因果関係

生成臨床UIが医療者に価値をもたらすまでには、少なくとも三つの条件がある。まず、専門家が診療シナリオに必要な質問、情報、完了条件を定める。次に、システムがその情報要求を正しいデータ取得と表示に変換する。最後に、医療者が表示を使って必要な情報を確認し、診療タスクを行う。本稿が直接測るのは二番目である。

層間整合性が高ければ確認負担や見落としが減るとは、本結果だけでは言えない。層間整合性は、妥当な情報を使ったUXを評価する前提になる。対象患者や期間を誤った画面を使って、操作時間や好みだけを測っても、診療タスクに適したUIかは判断できないためである。今後は、専門家が確認したシナリオと合成症例を使い、誤った更新の受理、妥当な更新の受理、問題箇所の特定、修復、確認時間、操作数、認知負荷を分けて測る必要がある。

3.3 何を情報の誤りと呼ぶか

情報の正しさには複数の段階がある。本稿では、元データの記録内容が事実か、診療シナリオのタスク定義が医学的に適切か、表示を見た医療者の判断が正しいかを評価しない。本稿が扱う**情報の誤り**は、定められた情報要求に対して、画面へ届いた情報が患者、項目、期間、集約、情報源、表示方法、接続、版のいずれかで一致しない状態である。層間不整合は、この誤りがどの成果物の間で起きたかを示す。

この定義により、SQLが実行できない失敗と、SQLは実行できるが別の情報を返す失敗を区別できる。前者は局所検査で見つけられる。後者を見つけるには、情報要求とSQL、SQLと実行結果、実行結果と表示の対応を検査しなければならない。

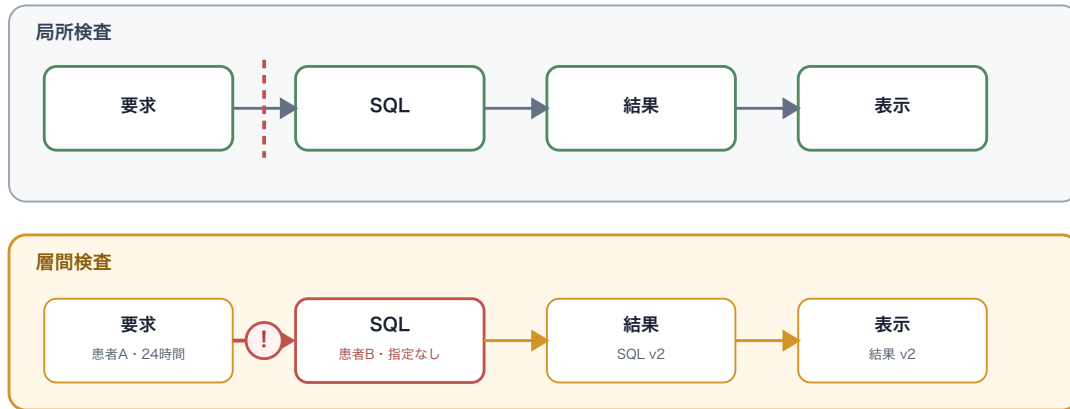


Fig. 2. 局所検査と層間検査の違い。上段では、要求、SQL、結果、表示がそれぞれ検査を通る一方、要求と SQL の患者と期間のずれは調べない。下段では、患者と期間を隣接する層の間で照合するため、要求と SQL の間にある不一致を特定できる。正しい値が一意に決まる場合は、その対応関係を使って一回だけ修復する。

4 層間整合性の定義

4.1 対象とする情報の流れと検知の仕組み

本研究は、情報要求、SQL、実行結果、表示部品の4層を扱う。情報要求は、対象患者、対象項目、期間、集約、情報源、期待する表示方法を含む。SQLは要求をデータベース操作に変換する。実行結果はSQL、入力、実行時点に対応する版を持つ。表示部品は、結果を参照し、その形状に適した方法で提示する。

図2に、層間不整合を見つける仕組みを示す。局所検査は各箱の内部だけを見る。層間検査は、情報要求から取り出した意味キーをSQLの解析結果と比べる。さらに、SQLと実行結果の版、結果と表示部品の参照先および形状も比べる。不一致があれば、満たさなかった述語と、その述語が結ぶ二つの成果物を返す。同じ意味を表す値を層の境界で比べることが検出を支えており、グラフ探索の性質によるものではない。

更新候補 c が層間整合性を満たすとは、要求とSQLの意味が一致し、SQLと実行結果の版が一致し、結果と表示部品の参照および表示方法が一致することとする。これを次の述語の論理積で表す。

$$I(c) = P(c) \wedge D(c) \wedge T(c) \wedge A(c) \wedge S(c) \wedge V(c) \wedge L(c) \wedge R(c) \quad (1)$$

P は患者、 D は項目、 T は期間、 A は集約、 S は情報源、 V は表示方法、 L はデータと表示の接続、 R はSQLと実行結果の版の一致を表す。妥当候補は8述語をすべて満たし、不整合候補は8述語のうち一つだけを満たさないように作る。

4.2 8種類の不整合

表1に不整合の定義を示す。対象患者の不整合では、要求側の患者を保ったままSQLの患者条件を別の患者を指す値に変える。対象項目では、要求と異なる列または集約対象を取得する。期間では、要求の時点や期間に対応する条件を変える。集約方法では、件数、最大、最小、平均などを別の演算へ置き換える。

情報源の不整合では、要求が対応付けられた表とSQLの参照表をずらす。表示方法では、SQL、実行結果、列、値を保ったまま、基準が数値カードならデータフレーム表示へ、基準がデータフレーム表示なら別の表部品へ変

Table 1. 評価で生成する 8 種類の層間不整合

種類	変更するもの	単独では通り得る検査
対象患者	SQL の患者条件	構文、読み取り専用、実行
対象項目	取得列または集約対象	構文、実行、結果型
期間	時点・期間条件	構文、実行
集約方法	件数、最大、最小、平均など	構文、実行、結果型
情報源	参照表	構文、実行
表示方法	結果を示す部品の種類	UI 構造、描画
接続	部品が参照するデータ取得	個別ノードの型
古い結果	SQL 更新後の結果版	SQL が動くか、結果型、描画

える。接続では、表示部品が別のデータノードを参照する。古い結果では、SQL を更新しても実行結果の版を以前のまま残す。

これらは臨床現場で観測したエラー頻度の分類ではない。どの関係を検査する必要があるかを測定可能にするための故障モデルである。各候補には一種類だけを注入するため、結果は複合故障への性能を示さない。

5 評価基盤

5.1 基準ケースの構築

入力は EHRSQL-2024 の回答可能な質問と正解 SQL である。回答不能ケースは正解 SQL から基準 UI を構築できないため、主評価から除外し、件数を記録する。正解 SQL は MIMIC-IV Demo v2.2 の SQLite データベースに対して読み取り専用で実行する。書き込みを含む文、複数文、許可しない構文は実行前に拒否し、SQL ごとの実行上限を 5 秒とした。

実行結果が単一値なら数値カード、複数行または複数列ならデータフレーム表示を基準とする。空結果は除外せず、列構造を持つ結果として扱う。基準ケースは、質問テンプレートから抽出した意味スロット、SQL の解析結果、結果形状、データノード、表示部品、各成果物の版を保持する。

同じ質問テンプレートと SQL 構造を共有するケースを更新前後として組み合わせる。この組から、すべての関係を同時に更新した妥当候補と、一つの関係だけを壊した不整合候補を作る。同じ質問形式から派生する候補は独立ではないため、後の統計処理では質問テンプレートを再標本化の単位とする。

5.2 独立したオラクル

候補生成と正解判定は別の処理で実装した。オラクルは、質問テンプレートの意味スロット、SQL の解析結果、実行結果の形状と版、データノードと表示部品の参照を照合する。候補生成器が付けた期待ラベルとオラクルの判定が一致しない場合、評価を停止する。

完成した JSON の完全一致を正解としない。安定 ID、順序、表示文言など、意味を変えない差を許すためである。一方、画面の見た目が同じでも、別患者、異なる期間、古い結果を参照する候補は不整合とする。この設計により、表層的な一致と意味上の対応を分ける。

5.3 4 種類の検査条件

同じ候補を表 2 の 4 条件に入力する。局所検査は、SQL が読み取り専用で実行できるか、成果物がスキーマに合うか、表示部品を描けるかを個別に確かめる。成果物内検査は、各成果物に患者や項目などを調べる規則を加え、一部の隣接層も照合する。ただし、情報要求から表示までの全関係は検査しない。

Table 2. 比較する 4 種類の検査条件

条件	個別成果物	層間関係	保存形式
局所検査	基本検査	なし	各成果物
成果物内検査	詳細検査	一部	各成果物
グラフ検査	詳細検査	すべて	ノードと辺
表形式検査	詳細検査	すべて	一枚の表

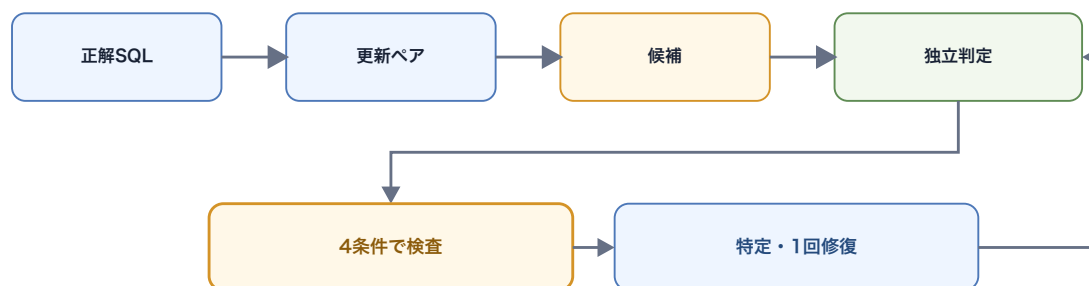


Fig. 3. 評価手順。正解 SQL から基準ケースを作り、同じ質問形式の更新前後を組にして妥当候補を作る。その対応関係を一つだけ変えて不整合候補を作る。候補生成とは別のオラクルで正解ラベルを確かめた後、同じ候補を局所検査、成果物内検査、グラフ検査、表形式検査に入力する。流出、妥当候補の受理、問題箇所特定、一回の修復を記録し、修復した候補もオラクルで確認する。

グラフ検査では、質問テンプレート、SQL、結果、データノード、表示部品をノードとし、患者、項目、期間、集約、情報源、版、参照、表示方法の対応を辺で表す。候補を更新した後、影響を受ける辺をもう一度調べる。表形式検査は、グラフ検査と同じ意味スロットと対応関係を一枚の表に持ち、同じ述語を調べる。二条件の内容をそろえ、検査内容による効果と保存形式による効果を分ける。

5.4 診断と一回の修復

検査が候補を拒否した場合、失敗した述語と関係する成果物を診断として返す。特定率は、注入した不整合の種類を診断が正しく示した割合とする。検出しただけで、場所または種類を誤った場合は特定成功に数えない。

修復では診断を一回だけ使い、基準となる対応関係から一意に決まる値に戻す。たとえば、表示部品の参照先、結果の版、表示方法は、対応する規則から復元する。修復した候補を、オラクルが用いる整合性述語でもう一度調べ、すべてを満たした場合だけ成功とする。探索、複数回の試行、生成モデルによる書き換えは行わない。

5.5 評価手順

図 3 に評価手順を示す。開発用データでは、異なる質問テンプレートを優先した 50 件で、SQL を実行して基準ケースに変換できることを確かめた。調整確認用データでは、不整合ごとの候補数、質問テンプレート数、ラベルの一致、機微情報が保存されないことを確かめた。列順の不整合を作れなかったため、テスト分割を見る前に表示方法の不整合に置き換えた。他の不整合、指標、分析、停止条件は維持した。

調整確認用データで事前基準を満たした後、評価コード、設定、分析手順を Git commit baa82075b6bf で固定した。固定後、テスト分割を一度だけ実行した。評価中に生成モデルや外部モデルは呼び出していない。

5.6 指標と統計分析

主指標は不整合流出率と妥当更新受理率である。不整合流出率は、注入した不整合候補のうち検査が受理した割合であり、低いほどよい。妥当更新受理率は、すべての関係を正しく更新した候補のうち受理した割合であり、高いほどよい。副次指標として、問題箇所の特定期率、一回の修復成功率、候補当たりの平均検証時間を測る。

条件間の差は、同じ候補に対する判定結果の差として計算する。95%信頼区間は、質問テンプレート単位のクラスターブートストラップを 10,000 回行って求める。再標本化の単位を質問テンプレートとするのは、同じ SQL 構造と変異規則を共有する派生候補を独立例として扱わないためである。

5.7 データ管理と再現性

公開成果物には、生データ、質問文、正解 SQL、患者 ID、患者単位の結果値を保存しない。保存するのは、ハッシュ化したケース識別子、入力と候補のチェックサム、不整合の種類、判定、診断、修復、集計、実行環境である。正式結果は JSON と JSON Lines で固定し、別の処理で TSV へ変換して件数、主要指標、95%信頼区間を再計算した。再集計値は元の集計と一致した。

6 結果

6.1 基準ケースと候補

テスト分割には 1,167 件があり、正解 SQL を持つ回答可能ケース 934 件を対象とした。回答不能 233 件は主評価から除外した。934 件すべてで正解 SQL の実行と基準成果物の構築に成功し、82 件は列構造を持つ空結果だった。134 種類の質問テンプレートから 763 組の更新前後ペアを作った。各ペアについて妥当候補と不整合候補を作り、4 条件に入力したため、検査回数は $763 \times 2 \times 4 = 6,104$ 回となった。候補ラベルと独立オラクルの不一致は 0 件だった。

作成した不整合は、対象患者 52 件、対象項目 55 件、期間 106 件、集約方法 34 件、情報源 134 件、表示方法 134 件、接続 129 件、古い結果 119 件の合計 763 件だった。患者、項目、期間、集約では、意味を一つだけ変えられる代替 SQL がない場合に候補化を見送った。候補を作れるかどうかはデータと質問形式に依存するため、8 種類の件数は均等ではない。

6.2 RQ1：局所検査を通過する不整合

局所検査は 8 種類すべての不整合候補を受理し、不整合流出率は 100.00%だった。各候補では、SQL を読み取り専用で実行でき、結果と表示部品も個別には有効になるよう構成している。この結果は、対象患者や期間の誤りに加え、表示方法、接続、古い結果も、構文、スキーマ、画面を描けるかどうかを個別に確かめるだけでは検出できないことを示す。

成果物内検査の流出率は、不整合の種類によって異なった。表 3 に示すように、情報源、表示方法、古い結果の不整合はすべて拒否した。一方、患者、項目、期間、集約の不整合は 92.73%から 98.08%が流出し、接続の不整合はすべて流出した。各成果物の中に書ける規則は検査できても、要求と SQL、データノードと表示部品のように成果物をまたぐ対応は十分に確かめられなかった。

6.3 RQ2：4 条件の検出、受理、特定

主結果を表 4 に示す。グラフ検査と表形式検査は、不整合候補 763 件をすべて拒否し、妥当候補 763 件をすべて受理した。局所検査との流出率の差は -100.00 ポイントで、95%信頼区間は $[-100.00, -100.00]$ だった。成果物内検

Table 3. 成果物内検査を通過した不整合の割合

種類	生成数	流出数	流出率
対象患者	52	51	98.08%
対象項目	55	51	92.73%
期間	106	103	97.17%
集約方法	34	33	97.06%
情報源	134	0	0.00%
表示方法	134	0	0.00%
接続	129	129	100.00%
古い結果	119	0	0.00%
合計	763	367	48.10%

Table 4. 4 条件の主結果と副次結果

条件	不整合流出	妥当受理	特定	平均時間
局所検査	763/763	763/763	0/763	2.152 ms
成果物内検査	367/763	763/763	387/763	7.441 ms
グラフ検査	0/763	763/763	763/763	9.339 ms
表形式検査	0/763	763/763	763/763	9.296 ms

査との差は -48.10 ポイントで、95%信頼区間は [-49.56, -46.53] だった。妥当更新受理率は 4 条件すべて 100.00% で、条件差は 0 だった。

成果物内検査は、不整合 396 件を拒否し、そのうち 387 件で注入箇所を正しく特定した。全不整合に対する特定率は 50.72% だった。グラフ検査と表形式検査は、763 件すべてで種類と箇所を特定した。両条件の判定不一致は 0 件であり、不整合流出率の差も 0 ポイントだった。候補当たりの平均検証時間は、局所検査 2.152 ms、成果物内検査 7.441 ms、グラフ検査 9.339 ms、表形式検査 9.296 ms だった。この評価環境では、二つの層間検査の時間差は 0.043 ms だった。

6.4 RQ3：一回の修復

局所検査は不整合を検出しないため、修復しなかった。成果物内検査は、拒否した 396 件を一回ずつ修復し、387 件が独立オラクルのすべての述語を満たした。修復を試みた候補に対する成功率は 97.73% である。残る 9 件は、一回の修復後も整合性を満たさなかった。

グラフ検査と表形式検査は、全 763 件の不整合を特定した。一回の修復後には、全件が整合性を満たした。両条件で不要な拒否はなく、妥当候補には修復を適用しなかった。この 100% は、単一故障、あらかじめ定めた対応関係、同じ入力には同じ結果を返す修復規則という本評価の範囲で得た結果であり、自然に生じる複合的な生成誤りにも当てはまるとは限らない。

7 考察

7.1 SQL の正しさから、要求した情報を表示へ届ける過程の正しさへ

局所検査では、8 種類すべての不整合が通過した。SQL、結果、表示部品がそれぞれ正常に動くことは必要だが、それだけでは医療者へ正しい情報を返せない。生成臨床 UI の評価単位は SQL や画面の単体ではなく、診療シナリオから情報要求を定め、データを抽出し、結果を表示する一連の過程である。

この枠組みでは、SQL 生成の改善は一連の過程の一部に位置付く。正しい SQL が一度得られても、増分更新で患者、期間、結果の版、表示先がずれれば、最終画面は要求を満たさない。逆に、見た目が整った画面も、取得元を確認できなければ妥当な UX とは判定できない。生成 UI 研究では、表層の品質、個別成果物が動くかどうか、情報の来歴、利用者が行う確認を分け、最後に同じ診療タスクへ結び直す必要がある。

7.2 層間検査が誤りを見つけた理由

成果物内検査は、情報源、表示方法、古い結果の不整合をすべて拒否したが、患者、項目、期間、集約の大半と、接続の不整合をすべて見逃した。前者は、許可する表、結果の形状、版などを一つの成果物内に書ける。後者は、要求と SQL、またはデータノードと表示部品を比べなければ分らない。

層間検査は、患者、項目、期間、集約、情報源、表示方法を意味キーとして持ち、版と参照先も含めて隣接する成果物の間で比べた。SQL をもう一度実行するだけでなく、要求から受け継ぐべき値が層をまたいでも同じかを判定した。不一致を対応関係の位置に結び付けるため、候補を拒否するだけでなく、問題箇所の特定と修復にも使えた。この仕組みを説明して初めて、流出率 0%という結果を解釈できる。

7.3 シナリオ差が示す医療固有の要件

麻酔科術前外来と甲状腺術後フォローの差は、臨床 UI に単一の正解表示を置きにくい理由を示す。術前外来では、広い患者背景と直近検査を一回の評価に集約する。甲状腺術後フォローでは、限られた検査と処方の時間変化を繰り返し比較する。同じ項目でも、最新値が必要な場面と推移が必要な場面があり、基準日も手術日と診察日で異なる。妥当な UI は、診療シナリオとタスクから決まる。

医療では、情報が表示されたかだけでなく、誰の、何の、いつの情報かが判断材料になる。別の患者、誤った期間、古い結果の表示は、画面の小さな不具合として扱えない。専門家が定めた必要情報、安全に関わる情報、完了条件と表示を結び付け、情報の来歴と更新時点を確かめられる設計が要る。ただし、本評価は二つのシナリオの医学的妥当性も、層間検査が臨床上の誤りを減らすかどうかを実証していない。シナリオは設計要件を具体化し、次の評価で専門家に何を確かめてもらうかを示す役割を持つ。

7.4 関係の内容が結果を分けた

グラフ検査と表形式検査は、判定、問題箇所の特定、一回の修復で同じ結果を得た。検出性能について言えるのは、8 種類の関係を明示して検査すれば、保存形式がグラフでも表でも同じ候補を拒否できたということまでである。グラフ形式そのものを安全性や性能の根拠にはできない。

この否定的結果は、中間表現を評価するときに、保存形式と利用者に提供する機能を分ける必要があることを示す。グラフは影響範囲の探索、部分更新の伝播、関係の可視化に使える可能性がある。本評価は、その編集しやすさや説明の分かりやすさを測っていない。今後の比較では、同じ最小単位の情報と修復操作を両条件に与え、層間の経路をグラフとして提示することが、医療者による問題箇所の特定と修復を支えるかを測るべきである。

Jelly、Bridging Gulfs、SimStep は、中間表現を利用者が意図を確かめ、生成物を編集する接点として用いている [1, 5, 8]。本研究は、こうした接点に保持した関係を、実行結果と表示まで検査できるようにすることを、デー

タ駆動型システムの設計要件として提案する。中間表現の価値は、その構造だけでは決まらず、利用者の確認とシステムの実行をどの関係で結ぶかにある。

7.5 検知を利用者価値へつなぐ

層間検査が誤りを見つけても、それだけで利用者に役立つとは言えない。本稿の結果から直接言えるのは、既知の単一不整合を表示前に止め、問題箇所を機械可読な形で返せたことまでである。この機能を医療者の作業に生かすには、診断を更新時の対話に組み込み、利用者が受理、拒否、部分修正を選べるようにする必要がある。

たとえば「無効な UI」とだけ示すより、「要求の患者と SQL の患者条件が一致しない」「表示部品が更新前の結果版を参照している」と示せば、利用者が確認する範囲を狭められるかもしれない。ただし、診断が確認時間を短くするか、誤った更新の受理を減らすか、妥当な更新まで過剰に拒否させないかは、人を対象とする比較で確かめなければならない。最終画面だけを示す条件と、同じ画面に層間の経路と診断を加える条件を比べ、不正更新の誤受理、妥当更新の受理、問題箇所の特定、意味的に正しい修復、作業時間、認知負荷を分けて報告する。

生成システムの提案を利用者が受理、拒否、交渉する過程を扱った研究 [9] を踏まえると、診断表示の評価では正答率だけをまとめてはいけな。不正更新と妥当更新を一つの正答率に混ぜず、不正更新の誤受理と妥当更新の受理を分けて測る。層間情報が識別を助けたのか、単に拒否を増やしたのかを区別するためである。

7.6 修復と人の判断を分ける

層間関係をすべて持つ二条件は、既知の単一故障 763 件を一回で修復した。患者条件、参照先、結果の版、表示方法など、対応関係から正しい値が一意に決まる場合は、変更箇所だけを機械的に戻せる。したがって、検査は候補を拒否するだけでなく、修正案も返せる。

自然言語の要求が曖昧な場合、臨床的に適切な表示の細かさを選ぶ場合、複数の成果物が同時に変わった場合は、修正方法が一つに決まらない。生成 UI は、機械的に直せる関係、複数の案を人に示す関係、専門家が決める関係を区別する必要がある。100%の修復成功率は、自動修復が一般に安全であることを示さない。既知の対応関係から正しい値が一意に決まる範囲だけを示している。

7.7 医療以外のデータ駆動型 UI への応用

本研究の考え方は、利用者の要求からデータを取得し、ダッシュボード、表、可視化を生成する他の分野にも応用できる可能性がある。たとえば金融では、指定した口座、銘柄、評価期間、通貨、価格の情報源と、表示内容との対応が一貫している必要がある。航空や設備保全では、対象となる機体や設備、観測期間、センサー、集約方法、データの版を追跡する必要がある。どの例でも、問い合わせを実行でき、画面を描けるだけでは、利用者が求めた対象、期間、情報源、版と一致するとは限らない。

一方、医療向けに定めた 8 種類の不整合は、そのまま他分野へは移せない。応用先では、利用者のタスクを検査できる項目に分けられること、データ取得から表示までの対応を記録できること、機械が調べる関係と人が判断する関係を分けられることが前提になる。金融なら価格の基準時刻や通貨、航空なら機体識別子やセンサーの校正状態など、その分野で誤りの意味を左右する項目を追加する必要がある。今後は、各分野の専門家と不整合の分類を作り直し、層間の診断表示が誤りの見落としや確認時間をどう変えるかを利用者評価で調べたい。CHI に対する本稿の主張は、データ取得を伴う生成 UI の評価では、利用者のタスクから最終表示までを追跡し、どこで対応が崩れたかを示す必要があるという点にある。

8 制約と倫理

第一に、本評価では、同じ入力から同じ不整合を作る規則を使い、一つずつ注入した。そのため、生成モデルが自然に起こすエラーの分布は表していない。単一故障で 100%を得た条件が、複合故障、曖昧な要求、未知の表示部品でも同じ性能を持つとは限らない。今後は、固定した検査を生成モデルが作った候補に適用し、今回の分類に含まれない失敗を質的に調べる必要がある。

第二に、正解 SQL を出発点としているため、Text-to-SQL モデルの初回生成誤りを含まない。実運用では、質問の解釈と SQL 生成も誤り得る。本結果は、正しい基準が得られた後の増分更新に範囲を限定する。質問から SQL を生成する段階を含める場合は、棄権、複数解釈、SQL の臨床的妥当性を別に評価する必要がある。

第三に、MIMIC-IV Demo v2.2 と EHRSQL-2024 の質問テンプレートだけを使った。麻酔科術前外来と甲状腺術後フォローは問題設定を説明する設計例であり、正式評価ケースではない。結果形状や利用可能な代替条件により、不整合ごとの候補数は異なる。他の診療シナリオ、データモデル、複数施設、複雑な可視化、複数データ源をまたぐ更新に結果を一般化できない。医療以外のデータや業務も評価していないため、他分野への応用例は今後確かめる設計上の見通しであり、実証結果ではない。

第四に、グラフ検査と表形式検査には、意図的に同じ対応内容を持たせた。この比較では保存形式だけを分けており、多数の関係を編集しやすいか、実装を保守しやすいか、説明を理解しやすいかは比べていない。検証時間も一つの実装と環境で測った値であり、形式一般の性能差として解釈できない。

第五に、本研究は匿名化済み公開デモデータ上の技術評価である。正解 SQL は技術的な基準であり、表示内容の医学的妥当性を専門家が確認した正解ではない。層間整合性が確認負担、誤った更新の受理、診療判断へ与える影響も測っていない。臨床上の安全性、診療判断の正確さ、使いやすさ、認知負荷、臨床転帰、実運用、他施設への一般化は評価していない。人を対象とする評価は、倫理審査と実施許可、募集、同意、撤回、データ管理を確認した後の別研究とする。

公開成果物には、質問文、正解 SQL、患者 ID、患者単位の結果値を含めない。MIMIC-IV Demo と EHRSQL-2024 の利用条件と出典を明記し、公開可能なチェックサム、分類、判定、集計だけを再現資料へ含める。

9 結論

診療シナリオに適した UI を生成するには、必要な情報を SQL で取得できるだけでなく、その要求が実行結果と表示まで保たなければならない。本研究は、この過程で起こる層間不整合を 8 種類に分け、公開デモデータ上で同じ入力から同じ不整合を作れるようにした。そして、同じ候補を 4 種類の検査条件で比較した。

763 組の更新前後ペアを使って 6,104 回検査したところ、局所検査は不整合をすべて受理し、成果物内検査も 48.10%を受理した。グラフ検査と、同じ内容を持つ表形式検査は、不整合をすべて拒否し、妥当な更新をすべて受理した。両者の結果に差がなかったため、本評価範囲では、検出を支えたのはグラフ形式そのものではない。患者、項目、期間、集約、情報源、表示方法、接続、版の対応を、実行できる規則として検査したことが、この結果につながったと解釈できる。

継続的に更新する生成 UI では、最終画面の見たと個別成果物の妥当性だけでなく、診療シナリオに基づく情報要求から表示までの来歴を調べ、問題箇所を説明できる必要がある。本研究は、妥当な臨床 UX を評価する前に満たすべき技術条件と、その検出の仕組みを示した。次の課題は、層間検査を専門家が確認したシナリオに適用し、診断表示が誤った更新の受理、問題箇所の特定、修復、確認負担に与える影響を、人を対象とする評価で確かめることである。さらに医療以外の分野では、その分野のタスクとデータに合わせて検査項目を定め直し、同じ設計原則が成り立つ範囲を調べる。

REFERENCES

- [1] Yining Cao, Peiling Jiang, and Haijun Xia. 2025. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 686, 20 pages. <https://doi.org/10.1145/3706598.3713285>
- [2] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View-Update Problem. *ACM Transactions on Programming Languages and Systems* 29, 3, Article 17 (2007). <https://doi.org/10.1145/1232420.1232424>
- [3] Xiao He, Zhenjiang Li, Zhenjiang Hu, Xiaoyu Liu, and Takashi Hirayama. 2018. Testing Bidirectional Model Transformation Using Metamorphic Testing. *Information and Software Technology* 104 (2018), 48–69. <https://doi.org/10.1016/j.infsof.2018.07.010>
- [4] Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Steven Horng, Leo Anthony Celi, and Roger Mark. 2023. MIMIC-IV. *PhysioNet* (2023). <https://doi.org/10.13026/6mm1-ek67> Version 2.2.
- [5] Zoe Kaputa, Anika Rajaram, Vryan Feliciano, Zhuoyue Lyu, Maneesh Agrawala, and Hariharan Subramonyam. 2026. SimStep: Human-in-the-Loop Authoring of Interactive Educational Simulations Through Task-Level Abstractions. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 75, 34 pages. <https://doi.org/10.1145/3772318.3791514>
- [6] Gyubok Lee, Sunjun Kweon, Seongsu Bae, and Edward Choi. 2024. Overview of the EHRSQL 2024 Shared Task on Reliable Text-to-SQL Modeling on Electronic Health Records. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*. Association for Computational Linguistics, Stroudsburg, PA, USA, 644–654. <https://doi.org/10.18653/v1/2024.clinicalnlp-1.62>
- [7] Damien Masson, Sylvain Malacria, G ry Casiez, and Daniel Vogel. 2024. DirectGPT: A Direct Manipulation Interface to Interact with Large Language Models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 975, 16 pages. <https://doi.org/10.1145/3613904.3642462>
- [8] Seokhyeon Park, Soohyun Lee, Eugene Choi, Hyunwoo Kim, Minkyu Kweon, Yumin Song, and Jinwook Seo. 2026. Bridging Gulfs in UI Generation through Semantic Guidance. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 880, 16 pages. <https://doi.org/10.1145/3772318.3791966>
- [9] Venkatesh Sivaraman, Leigh A. Bukowski, Joel Levin, Jeremy M. Kahn, and Adam Perer. 2023. Ignore, Trust, or Negotiate: Understanding Clinician Acceptance of AI-Based Treatment Recommendations in Health Care. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 754, 18 pages. <https://doi.org/10.1145/3544548.3581075>
- [10] Priyan Vaithilingam, Elena L. Glassman, Jeevana Priya Inala, and Chenglong Wang. 2024. DynaVis: Dynamically Synthesized UI Widgets for Visualization Editing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 217, 17 pages. <https://doi.org/10.1145/3613904.3642639>